“Programmer’s toolchest”

INF-1400 Objektorientert programmering

Martin H. Skjelvareid

This Photo by Unknown Author is licensed under CC BY-SA-NC

UiT The Arctic University of Norway

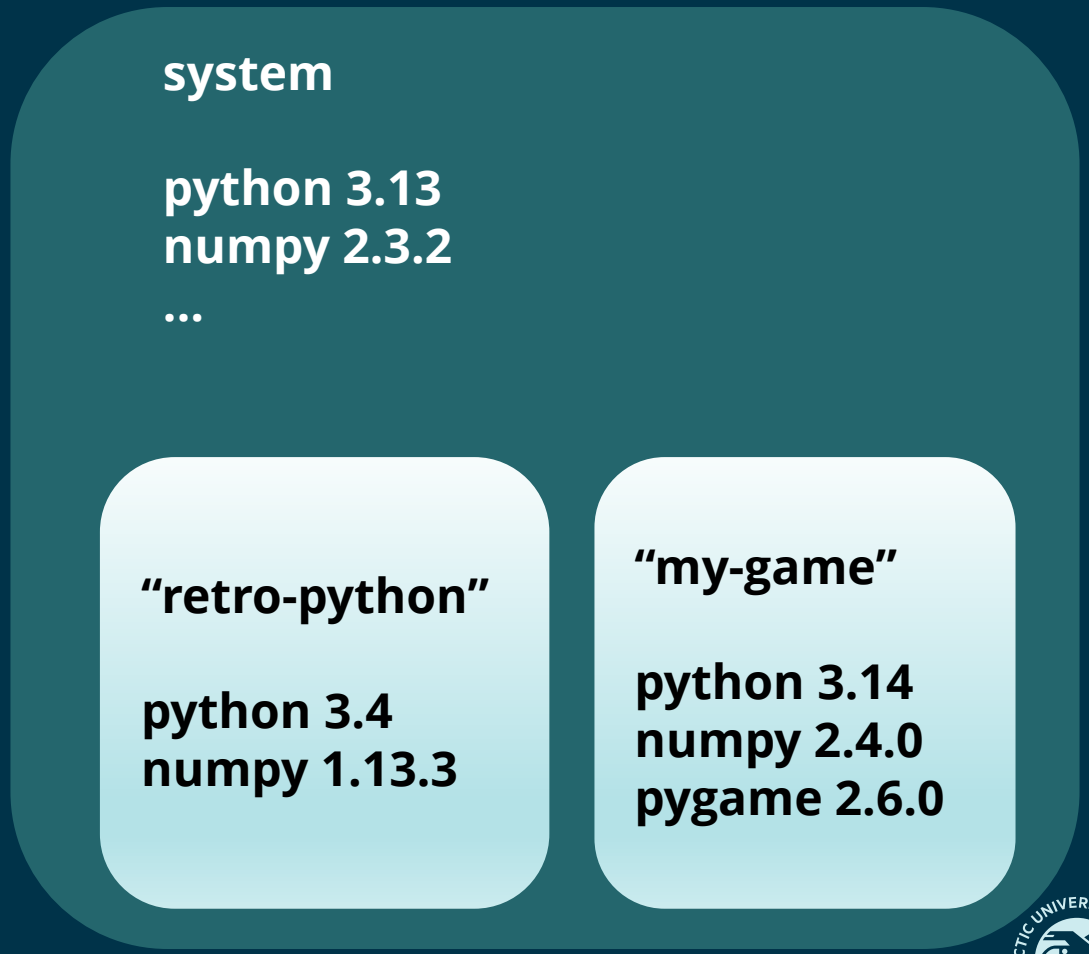
Temaer

- Virtual environments og prosjektorganisering
- Type hints
- Dokumentasjon:
  - Kommentarer i kode
  - Docstrings
  - Bygge dokumentasjon som statisk nettside
- Verktøy for å sjekke og formatere kode

Prosjektorganisering og virtual environments

Virtual environments (1)

- Hensikt: “Isolere” ulike utviklingsmiljøer fra hverandre
  - Ulike Python-versjoner
  - Ulike versjoner av eksterne pakker (f.eks. pygame, numpy, osv.)
- Oversikt over avhengigheter
- Risikohåndtering
  - Unngå sammenblanding med system-python
  - “Låse” pakker til versjoner som er kompatible med hverandre
  - Rekonstruere eksakt samme miljø på en annen maskin

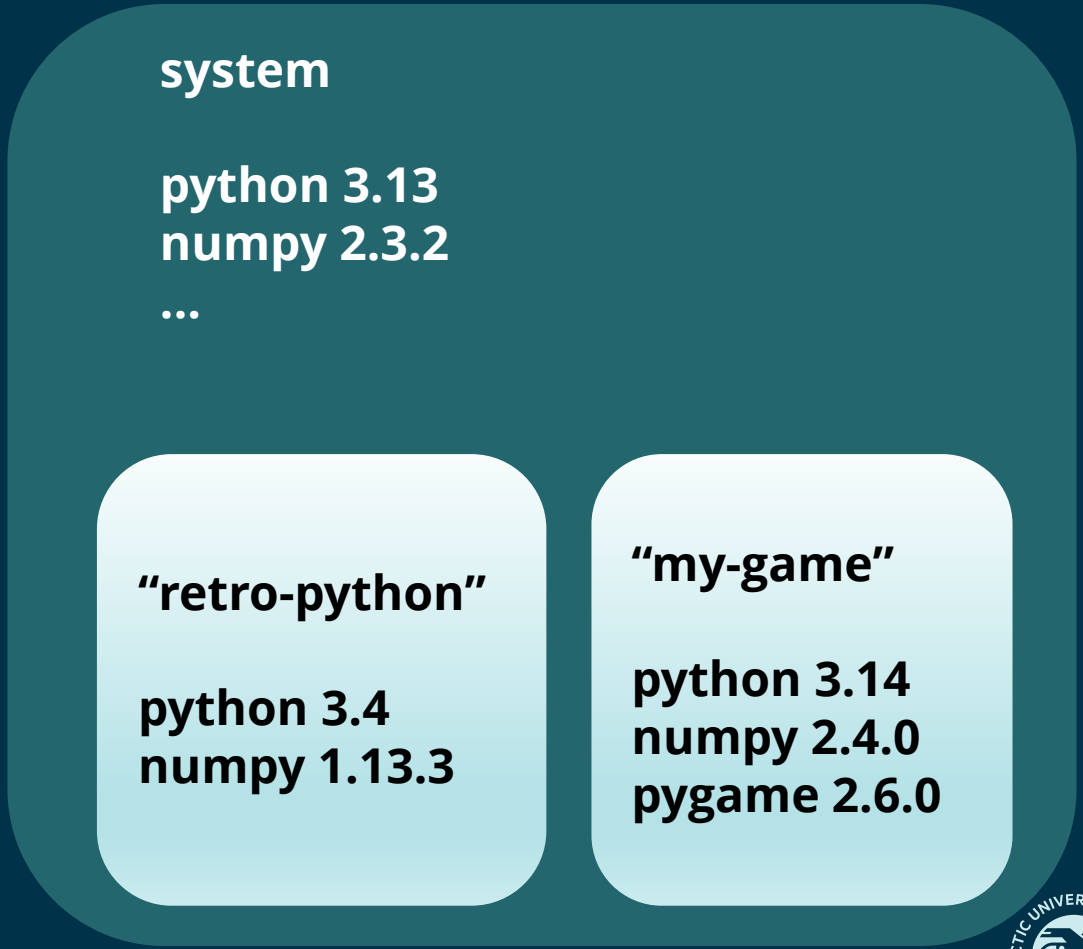


Virtual environments – ulike varianter (2)

- venv
  - standard, distribuert med Python
  - “lightweight” – unngå kopier om mulig
  - .venv-mappe:
    - Python: kopi eller “symlink”
    - Site-packages: Python-pakker
    - Ofte plassert i samme rot-mappe som kode
- Conda
  - Populær for «data science»
  - Kan inkludere programvare som ikke er Python (f.eks. CUDA, GDAL, FFMPEG, …)
  - Alle miljø «bor» i en egen conda-mappe, holdes utenfor kode
- Andre: virtualenv, poetry, …

Virtual environments – bruk (venv)

- Lage
  - python -m venv .venv
- Aktivere
  - .\.venv\Scripts\activate
- Installere pakker
  - Pakker installert med f.eks. pip blir automatisk del av miljøet
- Deaktivere
  - deactivate
- IDE (f.eks. VSCode)
  - Automatisk aktivering



Rekonstruere et miljø eksakt

- requirements.txt (pip)
  - pip freeze > requirements.txt
  - pip install –r requirements.txt
  - Eksakte versjoner for alle pakker
- pyproject.toml + lock-fil (UV)
  - “Førstehånds” avhengigheter i pyproject.toml
  - Transitive (“andrehånds”) avhengigheter i lock-fil
  - Rekonstruere (f.eks. etter git clone)
    - uv sync

```
asttokens==3.0.1
babel==2.18.0
backrefs==6.2
certifi==2026.2.25
charset-normalizer==3.4.4
click==8.3.1
colorama==0.4.6
comm==0.2.3
contourpy==1.3.3
cycler==0.12.1
debugpy==1.8.20
decorator==5.2.1
executing==2.2.1
fonttools==4.61.1
ghp-import==2.1.0
griffelib==2.0.0
idna==3.11
ipykernel==7.2.0
ipython==9.10.0
```

Eksempel: Del av requirements.txt

```
dependencies = [
    "ipykernel>=7.2.0",
    "matplotlib>=3.10.8",
    "mkdocs-material>=9.7.3",
    "mkdocstrings-python>=2.0.3",
    "numpy>=2.4.2",
    "requests>=2.32.5",
]
```

Eksempel: Del av pyproject.toml

UV – moderne alternativ til pip, venv, ++

- Raskt (implementert i Rust)
- Smart “caching”
- Opprette et nytt prosjekt
  - uv init
- Opprette et virtual environment (.venv)
  - uv venv
- Legge til avhengigheter
  - uv add numpy pygame
  - Oppdaterer pyproject.toml og uv.lock
- Fjerne avhengigheter
  - uv remove pygame
- “Synkronisere” (installere/oppdatere pakker)
  - uv sync

THE ARCTIC UNIVERSITY OF NORWAY • UiT •

Python type hints

Type hints – eksempel

- Innført i Python 3.5 (2015)
- Syntaks
  - variabelnavn : type (= verdi)
  - Kan nøstes (f.eks. list[int])
- Brukt bl.a. i funksjonssignatur
  - Inputparametre
  - Retur-verdi (-> type)
- Ingen direkte innvirkning på koden

```python
class TideInfo:
    def __init__(
        self,
        latitude: float,
        longitude: float,
        figure_size: tuple[float, float] = (10, 5),
    ):
```

```python
def _request_tide_info(
    self,
    latitude: float,
    longitude: float,
    start_time: str | None,
    end_time: str | None,
    datatype: Literal["pre", "tab"] = "pre",
) -> str:
```

Hvorfor bruke type hints?

- Del av dokumentasjonen – redusere uklarhet
- Oppdage bugs tidlig
  - Verktøy: pyright, mypy
  - Varsel når variabel ikke matcher forventet type
- Forbedrer IDE-verktøy (autocomplete++)
- Også anvendt i validering av datasett
  - Pydantic, FastAPI

```python
def concatenate(a, b):
    # "Concatenate" possible for many data types
```

```python
def concatenate_str(a: str, b: str) -> str:
    return a + b
```

```python
def concatenate_int(a: int, b: int) -> int:
    return int(str(a) + str(b))
```

```python
class User(BaseModel):
    id: int
    name: str = 'John Doe'
    signup_ts: datetime | None
    tastes: dict[str, PositiveInt]


external_data = {
    'id': 123,
    'signup_ts': '2019-06-01 12:22',
    'tastes': {
        'wine': 9,
        b'cheese': 7,
        'cabbage': '1',
    },
}
```

Eksempel: Datavalidering med pydantic

Fleksibel typing

- Statisk typede språk krever spesifikasjon av alle variabler
- Python mer fleksibel (på godt og vondt)
- Pyright type checking modes:
  - Off
  - Basic
  - Standard
  - Strict

Dokumentasjon

Ulik dokumentasjon for ulike publikum

- Nettside, dokument – til “alle”
  - Beskrivelse, eksempler, tutorials, “how-to”
  - API-referanse: Lang liste av klasse- og funksjons-signaturer
- Docstrings – til eksterne programmerere:
  - Beskrivelse av grensesnitt (input-parametere, retur-verdier, ++)
  - Brukes til å generere API-referanse
  - Dokumentasjon integrert i IDE
- Kommentarer – til «interne»:
  - «Påminnelse»/forklaring til deg selv eller andre programmerere

Pygame Front Page

Quick start

Welcome to pygame! Once you've got pygame installed (`pip install pygame` or `pip3 install pygame` for most people), the next question is how to get a game loop running. Pygame, unlike some other libraries, gives you full control of program execution. That freedom means it is easy to mess up in your initial steps.

Here is a good example of a basic setup (opens the window, updates the screen, and handles events)--

```
# Example file showing a basic pygame "game loop"
import pygame

# pygame setup
pygame.init()
screen = pygame.display.set_mode((1280, 720))
clock = pygame.time.Clock()
running = True

while running:
    # poll for events
    # pygame.QUIT event means the user clicked X to close your window
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

`pygame.display.get_window_size()`

Return the size of the window or screen

`get_window_size() -> tuple`

Returns the size of the window initialized with `pygame.display.set_mode()`. This may differ from the size of the display surface if SCALED is used.

```
try:
    leftradius = left.radius
except AttributeError:
    leftrect = left.rect
    # approximating the radius of a square by using half of the diagonal,
    # might give false positives (especially if its a long small rect)
    leftradius = 0.5 * ((leftrect.width**2 + leftrect.height**2) ** 0.5)
    # store the radius on the sprite for next time
    left.radius = leftradius
```

Fra pygame/src_py/sprite.py

Kommentarer i kode

- Bruk av funksjons- og variabelnavn bør gjøre kode "selvforklarende"
- Ingen grunn til å beskrive hver kodelinje
- Bruk kommenterer for å
  - Forklare ikke-intuitiv kode
  - Overskrifter
  - "Tagging" (f.eks. TODO)
- Men: Ingen harde regler, avhengig av "kultur"

```python
def parse_tide_time_series(xml_data: str) -> tuple[list[str], list[float]]:

    # Get water level data from XML
    xml_root = ET.fromstring(xml_data) # Parse XML string into ElementTree object
    waterlevels = xml_root.findall(".//data/waterlevel")

    # Loop over waterlevel elements, save timestamps and values to lists
    time = []
    level_values = []
    for waterlevel in waterlevels:
        timestamp = waterlevel.attrib.get("time")
        value = waterlevel.attrib.get("value")
        if timestamp and value:
            time.append(timestamp)
            level_values.append(float(value))

    return time, level_values
```

Docstrings

- Dokumentasjon av moduler, klasser, metoder, funksjoner
- Generelle “docstring conventions”: PEP 257
- Spesifikke stiler:
  - Google
  - NumPy / SciPy
  - reStructuredText
  - ++

```python
def request_tide_info_numpy(latitude: float, longitude: float) -> str:
    """Request tide information from the API.

    Parameters
    ----------
    latitude : float
        Latitude of the location (decimal degrees).
    longitude : float
        Longitude of the location (decimal degrees). Example: A long line of
        text is wrapped with the same indent as above.

    Returns
    -------
    str
        The raw XML response from the tide API.
    """
```

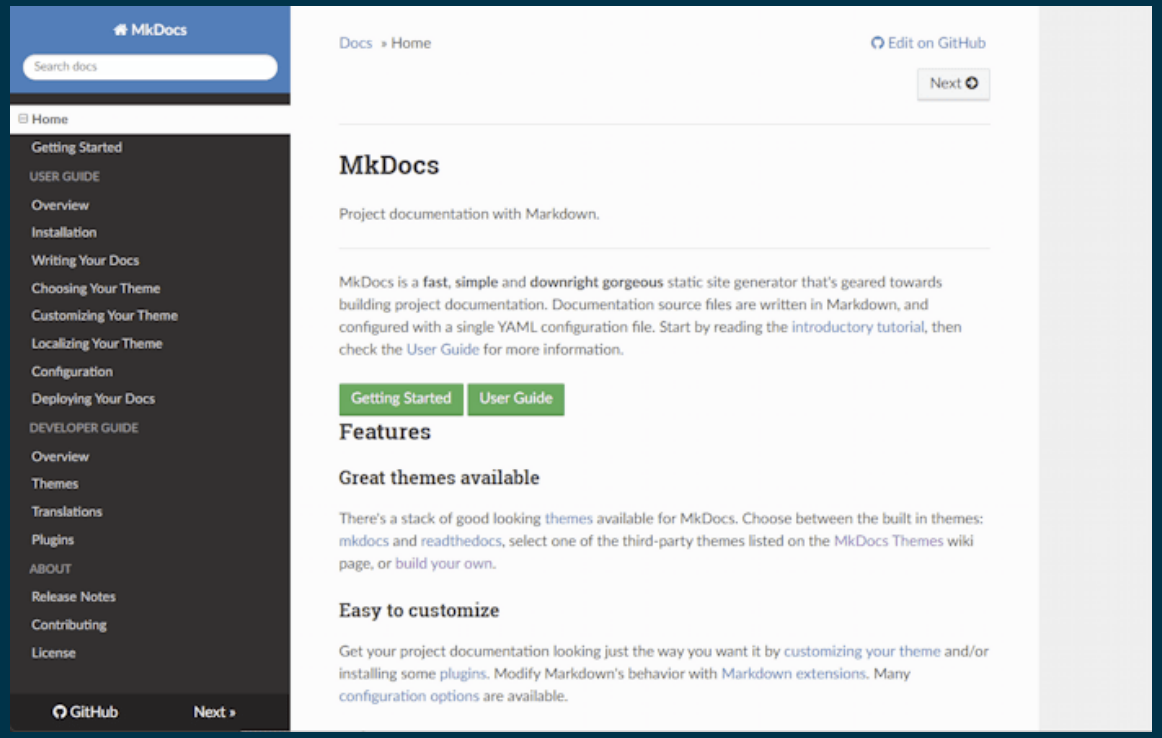
```python
def request_tide_info_google(latitude: float, longitude: float) -> str:
    """Request tide information from the API.

    Args:
        latitude (float): Latitude of the location (decimal degrees).
        longitude (float): Longitude of the location (decimal degrees). Example: A
            long line of text is wrapped with an extra indent.

    Returns:
        str: The raw XML response from the tide API.
    """
```

MkDocs

- Verktøy for å generere statisk nettside for dokumentasjon
- Innhold skrives i MarkDown / hentes fra docstrings
- Konfigurasjon i en enkelt YAML-fil
- Live HTML-server
  - mkdocs serve
- Bygge nettside (HTML)
  - mkdocs build
- Laste opp til github pages
  - mkdocs gh-deploy



Verktøy for å sjekke og formatere kode

Verktøy

- Mange verktøy utviklet for ulike oppgaver
  - black – autoformatering (PEP 8)
  - Flake8 – rapportere stil-problemer (PEP 8)
  - isort – sortere imports (PEP 8)
  - pydocstyle – sjekke docstring-formatering (PEP 257)
- Ruff
  - Ett enkelt verktøy som kombinerer alle over (m.fl.)
- Verktøy kan kjøres automatisk (ved lagring / commit)

```
import matplotlib.pyplot as plt
import requests
from numpy.typing import NDArray
import numpy as np
from typing import Literal
import xml.etree.ElementTree as ET
from datetime import datetime, timedelta
```

Usorterte import statements

```
import xml.etree.ElementTree as ET
from datetime import datetime, timedelta
from typing import Literal

import matplotlib.pyplot as plt
import numpy as np
import requests
from numpy.typing import NDArray
```

Sorterte import statements
- Interne pakker øverst
- Alfabetisk
- Skille import og from

Hvorfor bruke auto-fomatering

- Samme kode kan formateres på flere ulike måter
  - Irrelevante diffs
  - Krangel med andre om korrekt stil
- Ved autoformatering vil kun reelle endringer vises som diff

```python
def request_tide_info(latitude: float,longitude: float) -> str:
    """Request tide information from the API."""
    base_url = "https://vannstand.kartverket.no/tideapi.php"
    params = {
        "lat": latitude,"lon": longitude, "refcode": "cd",
        "interval": 10,
        "dst": 0,  "tzone": 1,
        "tide_request": "locationdata",  }



    response = requests.get(base_url, params=params)
    response.raise_for_status()  # Raise an error for bad status codes
    return response.text
```

```python
def request_tide_info(latitude: float, longitude: float) -> str:
    """Request tide information from the API."""
    base_url = "https://vannstand.kartverket.no/tideapi.php"
    params = {
        "lat": latitude,
        "lon": longitude,
        "refcode": "cd",
        "interval": 10,
        "dst": 0,
        "tzone": 1,
        "tide_request": "locationdata",
    }

    response = requests.get(base_url, params=params)
    response.raise_for_status()  # Raise an error for bad status codes
    return response.text
```